

DESAFIO TÉCNICO

Estagiário(a) de Desenvolvimento Back-End

Conta Bancária — API RESTful para uma cooperativa de crédito

1. OBJETIVO

Avaliar os fundamentos da pessoa candidata a estágio: orientação a objetos em Java, construção de uma API REST com Spring Boot, persistência com JPA/Hibernate em banco relacional e aplicação de regras de negócio — refletindo o tipo de tarefa que ela encontrará no time.

O desafio foi desenhado para ser resolvido em **até 7 dias corridos**. Não é necessário (nem esperado) implementar tudo com perfeição — priorize clareza, código organizado e um sistema que funcione de ponta a ponta.

2. Contexto do desafio

Uma cooperativa de crédito deseja modernizar seu sistema de gerenciamento de contas bancárias. Você foi designado(a) para desenvolver uma **API RESTful usando Spring Boot e MySQL ou H2** que permita gerenciar correntistas, contas bancárias e operações financeiras.

O sistema deverá armazenar dados de clientes, contas e transações (depósitos, saques e rendimentos), além de aplicar regras de negócio específicas para **conta corrente** e **conta poupança**.

3. Escopo funcional obrigatório

Entidades principais

- **Correntista** (nome, documento, dados de contato)
- **Conta** (número, saldo, tipo) — especializada em **Conta Corrente** (com limite) e **Conta Poupança**
- **Transação** (tipo, valor, data, conta de origem)

Regras que o sistema deverá atender

1. Cada correntista pode ter **várias contas**.
2. **Conta Corrente** possui um limite, e o saque só pode ocorrer até o valor *saldo + limite*.
3. **Conta Poupança** não permite saque além do saldo.
4. **Depósito** deve aumentar o saldo e registrar a transação.
5. **Saque** deve reduzir o saldo e registrar a transação.
6. Endpoints REST para:
 - Cadastro e consulta de correntistas
 - Abertura e consulta de contas
 - Depósito e saque
 - Extrato / listagem de transações da conta

4. Requisitos técnicos obrigatórios

CAMADA	REQUISITO
Linguagem	Java 8
Framework	Spring Boot
API	REST, com uso correto de verbos e status HTTP
Banco de dados	MySQL ou H2
ORM	JPA / Hibernate (Spring Data JPA)
Versionamento	Repositório Git público (GitHub) com histórico de commits significativo (evite um único commit "código final")
Documentação	README com instruções de execução local e endpoints disponíveis

5. Diferenciais avaliados

Opcionais, mas valorizados. Implemente quantos fizerem sentido dentro do tempo disponível — não é necessário fazer todos.

- **Conta Poupança** — endpoint para aplicar **rendimento mensal**, com taxa informada como parâmetro. O rendimento deve atualizar o saldo e registrar a transação.
- **Conta Corrente** — endpoint que aplica **juros sobre saldo negativo**, com taxa informada como parâmetro, e registra a transação.
- **Testes unitários** cobrindo as regras de saque, depósito e cálculo de rendimento/juros.
- Documentação da API com **Swagger / OpenAPI**.

- Tratamento de erros padronizado (mensagens claras e status HTTP adequados).

Se optar por não implementar algum diferencial, é totalmente aceitável descrever no README **como você faria** e por quê — isso também é avaliado.

6. Critérios de avaliação

- **Domínio de Orientação a Objetos** — correção na aplicação de herança, encapsulamento e abstração.
- **Design da API** — organização das camadas (Controller, Service, Repository).
- **Uso do Spring Data JPA** — mapeamento correto das entidades e relacionamentos.
- **Regras de negócio** — cumprimento das validações e cálculos definidos.
- **Boas práticas** — código limpo, nomes claros, padrões REST.
- **Uso de Git** (commits incrementais e com mensagens claras).
- **Documentação** (um dev externo consegue rodar o projeto seguindo o README?).
- **Diferenciais implementados** (pontuação extra, não eliminatório).
- **Comunicação**: clareza ao apresentar o que foi feito, o que não deu tempo, e por quê.

7. Entregáveis

1. Link do repositório público no GitHub, com README contendo:
 - Passos para rodar o projeto localmente.
 - Script SQL de criação do banco (ou `schema.sql`).
 - Endpoints disponíveis e exemplos de requisições (JSON).
 - O que foi feito, o que ficou de fora e por quê.
2. Opcional: um vídeo curto (3–5 min) demonstrando a API funcionando.

8. Etapa seguinte

Após a entrega, será marcada uma conversa técnica para revisar o código junto com você, entender as decisões tomadas e conversar sobre como a solução evoluiria em um cenário real.

O objetivo não é "quantos diferenciais você conseguiu marcar", e sim qualidade de raciocínio e código nas partes obrigatórias — não é obrigatório ter experiência com testes, Swagger ou cálculos financeiros, por exemplo.